INVERTIS UNIVERSITY BAREILLY

DBMS & SQL LAB (BCS-552)

B.Tech CSE 5th SEMESTER

SUBMMITED TO:

MR. RASHAN RAJ

SUBMMITED BY:

MANKIRAT SINGH
ID: BCS2024542
ROLL NO: 2410303257

Lab Assignment 1: Creating a Table and Adding Rows

Aim:

Create a table named student, insert three rows using one INSERT statement, and display all rows.

Software / Tools Required:

SQL database system (such as MySQL, PostgreSQL, or SQLite where supported), SQL editor/IDE, and a computer.

Theory:

A table stores data in rows and columns. CREATE TABLE defines the table structure, INT stores whole numbers, VARCHAR(30) stores text up to 30 characters, INSERT adds records, and SELECT retrieves records.

Algorithm / Procedure:

1. Open a SQL database tool and select or create a working database.
2. Create the student table with roll_no, name, and marks columns.
3. Insert the three supplied student records using one INSERT statement.
4. Run SELECT * to display every column and row.

Program / Queries / Commands:

```
CREATE TABLE student (     roll_no INT,     name VARCHAR(30),
marks INT
 );

 INSERT INTO student (roll_no, name, marks)
 VALUES
     (101, 'Asha', 78),
     (102, 'Rohan', 64),
     (103, 'Neha', 88);

 SELECT * FROM student;
```

Input:

(101, 'Asha', 78), (102, 'Rohan', 64), (103, 'Neha', 88)

Output:

Expected output (column spacing and display format may differ in your SQL tool):

roll_no	name	marks
101	Asha	78
102	Rohan	64
103	Neha	88

Result:

The SQL statements for Experiment 1 were prepared to perform the stated tasks. On executing the setup and queries successfully, the expected results shown above should be obtained.

Viva-Voce Questions:

1. What is a table in DBMS?
2. What is the purpose of CREATE TABLE?
3. What is the difference between INT and VARCHAR?
4. What does SELECT * return?

5. Lab Assignment 2: Updating and deleting Rows

Aim:

Update values in selected rows, increase a quantity, delete a row, and verify the table after each operation.

Software / Tools Required:

SQL database system (such as MySQL, PostgreSQL, or SQLite where supported), SQL editor/IDE, and a computer.

Theory:

UPDATE modifies existing records. The WHERE clause identifies which row should change. DELETE removes records that match its WHERE condition. SELECT is used to verify the changes.

Algorithm / Procedure:

1. Create the stock table and insert the three setup rows.
2. Update item_code 1 so its quantity becomes 80; display the table.
3. Add 10 to the quantity of item_code 2; display the table.
4. Delete item_code 3; display the remaining rows.

Program / Queries / Commands:

```
CREATE TABLE stock (    item_code INT,
item_name VARCHAR(30),    quantity INT
);

INSERT INTO stock (item_code, item_name, quantity)
VALUES
    (1, 'Notebook', 50),
    (2, 'Pen', 120),
    (3, 'Bag', 0);

-- Task 1
UPDATE stock
SET quantity = 80
WHERE item_code = 1;
SELECT * FROM stock ORDER BY item_code;

-- Task 2
UPDATE stock
SET quantity = quantity + 10
WHERE item_code = 2;
SELECT * FROM stock ORDER BY item_code;

-- Task 3
DELETE FROM stock
WHERE item_code = 3;
SELECT * FROM stock ORDER BY item_code;
```

Input:

Setup rows: (1, 'Notebook', 50), (2, 'Pen', 120), (3, 'Bag', 0).

Output:

Expected output (column spacing and display format may differ in your SQL tool):

Stage	item_code	item_name	quantity
After Task 1	1	Notebook	80
	2	Pen	120
	3	Bag	0
After Task 2	1	Notebook	80
	2	Pen	130
	3	Bag	0
After Task 3	1	Notebook	80
	2	Pen	130

Result:

The SQL statements for Experiment 2 were prepared to perform the stated tasks. On executing the setup and queries successfully, the expected results shown above should be obtained.

Viva-Voce Questions:

1. What is the use of UPDATE?
2. Why is WHERE important in UPDATE and DELETE?
3. What happens if DELETE is run without WHERE?
4. How can a value be increased using UPDATE?

Lab Assignment 3: Selecting Columns and Rows with WHERE

Aim:

Display selected columns and retrieve only records that satisfy specified conditions.

Software / Tools Required:

SQL database system (such as MySQL, PostgreSQL, or SQLite where supported), SQL editor/IDE, and a computer.

Theory:

SELECT chooses the columns to display. WHERE filters rows according to a condition. Comparison operators such as > and = are used to test values.

Algorithm / Procedure:

1. Create the student table and insert the four given records.
2. Select only name and marks.
3. Select all columns for students whose marks are greater than 60.
4. Select the name of the student whose roll number is 103.

Program / Queries / Commands:

```
CREATE TABLE student (    roll_no INT,
name VARCHAR(30),     marks INT
);

INSERT INTO student (roll_no, name, marks)
VALUES
    (101, 'Asha', 78),
    (102, 'Rohan', 64),
    (103, 'Neha', 88),
    (104, 'Deepak', 52);

-- Task 1
SELECT name, marks FROM student;

-- Task 2
SELECT * FROM student
WHERE marks > 60;

-- Task 3
SELECT name FROM student WHERE
roll_no = 103;
```

Input:

(101, 'Asha', 78), (102, 'Rohan', 64), (103, 'Neha', 88), (104, 'Deepak', 52)

Output:

Expected output (column spacing and display format may differ in your SQL tool):

Task 1: name	marks	
Asha	78	
Rohan	64	
Neha	88	
Deepak	52	
Task 2: roll_no	name	marks
101	Asha	78
102	Rohan	64
103	Neha	88
Task 3: name		
Neha		

Result:

The SQL statements for Experiment 3 were prepared to perform the stated tasks. On executing the setup and queries successfully, the expected results shown above should be obtained.

Viva-Voce Questions:

1. What is the purpose of SELECT?
2. What does WHERE do?
3. What is the difference between SELECT * and SELECT name?
4. Which operator checks whether marks are greater than 60?

1. *Lab Assignment 4: Combining Conditions*

Aim:

Combine conditions using AND, filter a range using BETWEEN, and check a list of values using IN.

Software / Tools Required:

SQL database system (such as MySQL, PostgreSQL, or SQLite where supported), SQL editor/IDE, and a computer.

Theory:

AND returns rows only when both conditions are true. BETWEEN checks whether a value lies within an inclusive range. IN checks whether a value matches any value in a supplied list.

Algorithm / Procedure:

1. Create the player table and insert the four supplied players.
2. Use AND to find Eagles players with more than 35 runs.
3. Use BETWEEN to find players with runs from 40 through 80.
4. Use IN to find players in the Tigers or Lions teams.

Program / Queries / Commands:

```
CREATE TABLE player ( name
VARCHAR(30), team VARCHAR(20),
runs INT );

INSERT INTO player (name, team, runs)
VALUES
    ('Asha', 'Eagles', 45),
    ('Meera', 'Tigers', 72),
    ('Arjun', 'Eagles', 30),
    ('Kiran', 'Lions', 88);

-- Task 1
SELECT name, runs FROM player
WHERE team = 'Eagles' AND runs > 35;

-- Task 2
SELECT name, runs FROM player
WHERE runs BETWEEN 40 AND 80;

-- Task 3
SELECT name, team FROM player WHERE
team IN ('Tigers', 'Lions');
```

Input:

('Asha','Eagles',45), ('Meera','Tigers',72), ('Arjun','Eagles',30), ('Kiran','Lions',88)

Output:

Expected output (column spacing and display format may differ in your SQL tool):

Task 1: name	runs
Asha	45
Task 2: name	runs
Asha	45
Meera	72
Task 3: name	team
Meera	Tigers
Kiran	Lions

Result:

The SQL statements for Experiment 4 were prepared to perform the stated tasks. On executing the setup and queries successfully, the expected results shown above should be obtained.

Viva-Voce Questions:

1. What is the function of AND?
2. Is BETWEEN inclusive of its boundary values?
3. What is the use of IN?
4. Can more than two conditions be combined in WHERE?

5. *Lab Assignment 5: Pattern Matching with LIKE*

Aim:
Find book titles that match specified text patterns using the LIKE operator.

Software / Tools Required:
SQL database system (such as MySQL, PostgreSQL, or SQLite where supported), SQL editor/IDE, and a computer.

Theory:
LIKE is used with WHERE for pattern matching. The percent sign (%) matches any sequence of characters, including no characters. The underscore (_) matches exactly one character.

Algorithm / Procedure:
1. Create the book table and insert the four book records.
2. Use 'Data%' to find titles starting with Data.
3. Use '%C' to find titles ending with C.
4. Use '_h%' to find titles whose second character is h.

Program / Queries / Commands:
```
CREATE TABLE book ( title
VARCHAR(40), category VARCHAR(20),
copies INT
);

INSERT INTO book (title, category, copies)
VALUES
    ('Database Systems', 'Computer', 3),
    ('Digital Electronics', 'Electronics', 2),
    ('Data Structures in C', 'Computer', 2),
    ('Short Stories', 'Literature', 1);

-- Task 1
SELECT title FROM book
WHERE title LIKE 'Data%';

-- Task 2
SELECT title FROM book
WHERE title LIKE '%C';

-- Task 3
SELECT title FROM book WHERE
title LIKE '_h%';
```

Input:
Four supplied titles: Database Systems, Digital Electronics, Data Structures in C, Short Stories.

Output:

Expected output (column spacing and display format may differ in your SQL tool):

Task 1: title
Database Systems
Data Structures in C
Task 2: title
Data Structures in C
Task 3: title
Short Stories

Result:

The SQL statements for Experiment 5 were prepared to perform the stated tasks. On executing the setup and queries successfully, the expected results shown above should be obtained.

Viva-Voce Questions:

1. What is the purpose of LIKE?
2. What does % represent in a LIKE pattern?
3. What does _ represent?
4. Which pattern finds values ending in C?

5. *Lab Assignment 6: Sorting Rows and removing Repeats*

Aim:
Sort product rows, display unique categories, and show the two most expensive products.

Software / Tools Required:
SQL database system (such as MySQL, PostgreSQL, or SQLite where supported), SQL editor/IDE, and a computer.

Theory:
ORDER BY sorts query results. DESC sorts from highest to lowest. DISTINCT removes duplicate values from the result. LIMIT restricts the number of returned rows in supported SQL systems.

Algorithm / Procedure:
1. Create the product table and insert the four products.
2. Sort product names and prices in descending price order.
3. Select distinct categories and sort them alphabetically.
4. Select the two highest-priced products using ORDER BY and LIMIT.

Program / Queries / Commands:
```
CREATE TABLE product (    product_name
VARCHAR(30),     category VARCHAR(20),
price INT
);

INSERT INTO product (product_name, category, price)
VALUES
    ('Pen', 'Stationery', 15),
    ('Notebook', 'Stationery', 60),
    ('Calculator', 'Electronics', 850),
    ('Bag', 'Office', 500);

-- Task 1
SELECT product_name, price FROM product
ORDER BY price DESC;

-- Task 2
SELECT DISTINCT category FROM product
ORDER BY category;

-- Task 3
SELECT product_name, price FROM product
ORDER BY price DESC
LIMIT 2;
```

Input:
Pen—15, Notebook—60, Calculator—850, Bag—500.

Output:

Expected output (column spacing and display format may differ in your SQL tool):

Task 1: product_name	**price**
Calculator	850
Bag	500
Notebook	60
Pen	15
Task 2: category	
Electronics	
Office	
Stationery	
Task 3: product_name	price
Calculator	850
Bag	500

Result:

The SQL statements for Experiment 6 were prepared to perform the stated tasks. On executing the setup and queries successfully, the expected results shown above should be obtained.

Viva-Voce Questions:

1. What is ORDER BY used for?
2. What is the difference between ASC and DESC?
3. What does DISTINCT do?
4. What does LIMIT 2 mean?

5. *Lab Assignment 7: Using AggregateFunctions*

Aim:
Use aggregate functions to count sales rows, calculate a total, and find the highest amount.

Software / Tools Required:
SQL database system (such as MySQL, PostgreSQL, or SQLite where supported), SQL editor/IDE, and a computer.

Theory:
Aggregate functions calculate a single summary value from multiple rows. COUNT counts rows or non-NULL values, SUM adds numeric values, and MAX returns the largest value.

Algorithm / Procedure:
1. Create the sales table and insert the four supplied sales records.
2. Use COUNT(*) and alias the result as num_sales.
3. Use SUM(amount) and alias the result as total.
4. Use MAX(amount) and alias the result as highest.

Program / Queries / Commands:
```
CREATE TABLE sales (      salesperson
VARCHAR(20),      region VARCHAR(20),
amount INT
);

INSERT INTO sales (salesperson, region, amount)
VALUES
    ('Deepak', 'North', 12000),
    ('Pooja', 'South', 8500),
    ('Deepak', 'South', 9500),
    ('Lata', 'North', 11000);

-- Task 1
SELECT COUNT(*) AS num_sales FROM sales;

-- Task 2
SELECT SUM(amount) AS total FROM sales;

-- Task 3
SELECT MAX(amount) AS highest FROM sales;
```

Input:
Amounts: 12000, 8500, 9500, 11000.

Output:

Expected output (column spacing and display format may differ in your SQL tool):

Task 1: num_sales
4
Task 2: total
41000

Task 3: highest
12000

Result:

The SQL statements for Experiment 7 were prepared to perform the stated tasks. On executing the setup and queries successfully, the expected results shown above should be obtained.

Viva-Voce Questions:

1. What is an aggregate function?
2. What does COUNT(*) return?
3. What does SUM() calculate?
4. What does MAX() return?

5. *Lab Assignment 8: Grouping Rows and using HAVING*

Aim:
Group sales by region, count records, calculate regional totals, and filter groups using HAVING.

Software / Tools Required:
SQL database system (such as MySQL, PostgreSQL, or SQLite where supported), SQL editor/IDE, and a computer.

Theory:
GROUP BY combines rows with the same value into groups. Aggregate functions can calculate values for each group. HAVING filters groups after aggregation, while WHERE filters individual rows before grouping.

Algorithm / Procedure:
1. Create shop_sale and insert the supplied records.
2. Group by region and count sales rows in each region.
3. Group by region and calculate the sum of amount.
4. Use HAVING to show only regions whose total is greater than 20000.

Program / Queries / Commands:

```
CREATE     TABLE     shop_sale     (
salesperson  VARCHAR(20),       region
VARCHAR(20),     amount INT
);

INSERT INTO shop_sale (salesperson, region, amount)
VALUES
    ('Deepak', 'North', 12000),
    ('Pooja', 'South', 8500),
    ('Deepak', 'South', 9500),
    ('Lata', 'North', 11000);

-- Task 1
SELECT region, COUNT(*) AS num_sales
FROM shop_sale
GROUP BY region
ORDER BY region;

-- Task 2
SELECT region, SUM(amount) AS total
FROM shop_sale
GROUP BY region
ORDER BY region;

-- Task 3
SELECT region, SUM(amount) AS total
FROM shop_sale
GROUP BY region
HAVING SUM(amount) > 20000
ORDER BY region;
```

Input:

North amounts: 12000 and 11000. South amounts: 8500 and 9500.

Output:

Expected output (column spacing and display format may differ in your SQL tool):

Task 1: region	num_sales
North	2
South	2
Task 2: region	total
North	23000
South	18000
Task 3: region	total
North	23000

Result:

The SQL statements for Experiment 8 were prepared to perform the stated tasks. On executing the setup and queries successfully, the expected results shown above should be obtained.

Viva-Voce Questions:

1. What is GROUP BY used for?
2. What is the purpose of HAVING?
3. How is HAVING different from WHERE?
4. Can aggregate functions be used with GROUP BY?

5. *Lab Assignment 9: Aliases and CASE*

Aim:
Rename result columns, classify students as Pass or Fail using CASE, and calculate marks with bonus marks.

Software / Tools Required:
SQL database system (such as MySQL, PostgreSQL, or SQLite where supported), SQL editor/IDE, and a computer.

Theory:
An alias gives a temporary name to a column or expression in query output. CASE evaluates conditions and returns a value for the first matching condition. Arithmetic expressions can be selected as calculated columns.

Algorithm / Procedure:
1. Create exam_result and insert the four names and marks.
2. Use AS to rename name and marks in the output.
3. Use CASE to show Pass when marks are at least 40, otherwise Fail.
4. Add 5 to marks and name the calculated column with_bonus.

Program / Queries / Commands:
```
CREATE TABLE exam_result (      name
VARCHAR(20),     marks INT
);

INSERT INTO exam_result (name, marks)
VALUES
    ('Asha', 92),
    ('Rohan',          47),
('Neha', 68),
    ('Meera', 33);

-- Task 1
SELECT name AS student_name, marks AS score
FROM exam_result;

-- Task 2 SELECT
name,
       CASE
           WHEN marks >= 40 THEN 'Pass'
           ELSE 'Fail'
       END AS result
FROM exam_result;

-- Task 3
SELECT name, marks + 5 AS with_bonus FROM
exam_result;
```

Input:
Asha—92, Rohan—47, Neha—68, Meera—33.

Output:

Expected output (column spacing and display format may differ in your SQL tool):

Task 1: student_name	score
Asha	92
Rohan	47
Neha	68
Meera	33
Task 2: name	result
Asha	Pass
Rohan	Pass
Neha	Pass
Meera	Fail
Task 3: name	with_bonus
Asha	97
Rohan	52
Neha	73
Meera	38

Result:

The SQL statements for Experiment 9 were prepared to perform the stated tasks. On executing the setup and queries successfully, the expected results shown above should be obtained.

Viva-Voce Questions:

1. What is a column alias?
2. What is the purpose of AS?
3. What does CASE do?
4. What happens in the ELSE branch?

5. *Lab Assignment 10: Joining two Tables with INNER JOIN*

Aim:
Join department and employee tables using their common dept_id column.

Software / Tools Required:
SQL database system (such as MySQL, PostgreSQL, or SQLite where supported), SQL editor/IDE, and a computer.

Theory:
A JOIN combines related records from two or more tables. INNER JOIN returns only rows that satisfy the join condition. Table aliases can make queries shorter and easier to read.

Algorithm / Procedure:
1. Create department and employee tables and insert the supplied records.
2. Join employee to department on dept_id and sort by employee name.
3. Filter the joined rows for the Sales department.
4. Display employee name, department name, and salary in descending salary order.

Program / Queries / Commands:

```
CREATE TABLE department (     dept_id
INT,     dept_name VARCHAR(20)
);

INSERT INTO department (dept_id, dept_name)
VALUES (10, 'Accounts'), (20, 'Sales'), (40, 'Research');

CREATE TABLE employee (
emp_name VARCHAR(30),
dept_id INT,     salary
INT
);

INSERT INTO employee (emp_name, dept_id, salary)
VALUES
    ('Anita', 10, 90000),
    ('Lata', 20, 55000),
    ('Deepak', 10, 60000),
    ('Pooja', 20, 42000);

-- Task 1
SELECT e.emp_name, d.dept_name
FROM employee AS e INNER JOIN
department AS d
    ON e.dept_id = d.dept_id
ORDER BY e.emp_name;

-- Task 2
SELECT e.emp_name FROM
employee AS e
INNER JOIN department AS d
```

```
    ON e.dept_id = d.dept_id
WHERE d.dept_name = 'Sales'
ORDER BY e.emp_name;

-- Task 3
SELECT e.emp_name, d.dept_name, e.salary
FROM employee AS e INNER
JOIN department AS d     ON
e.dept_id = d.dept_id ORDER
BY e.salary DESC;
```

Input:

Departments: (10, Accounts), (20, Sales), (40, Research). Employees: Anita/10/90000, Lata/20/55000, Deepak/10/60000, Pooja/20/42000.

Output:

Expected output (column spacing and display format may differ in your SQL tool):

Task 1: emp_name	**dept_name**	
Anita	Accounts	
Deepak	Accounts	
Lata	Sales	
Pooja	Sales	
Task 2: emp_name		
Lata		
Pooja		
Task 3: emp_name	dept_name	salary
Anita	Accounts	90000
Deepak	Accounts	60000
Lata	Sales	55000
Pooja	Sales	42000

Result:

The SQL statements for Experiment 10 were prepared to perform the stated tasks. On executing the setup and queries successfully, the expected results shown above should be obtained.

Viva-Voce Questions:

1. What is a join in SQL?
2. What does INNER JOIN return?
3. Why is dept_id used in the ON condition?
4. What is a table alias?

5. *Lab Assignment 11: Other kind of Joins*

Aim:
Use LEFT JOIN, CROSS JOIN, and NATURAL JOIN with team and player tables.

Software / Tools Required:
SQL database system (such as MySQL, PostgreSQL, or SQLite where supported), SQL editor/IDE, and a computer.

Theory:
LEFT JOIN returns all rows from the left table and matching rows from the right table; unmatched right-side columns are NULL. CROSS JOIN returns every possible pair of rows. NATURAL JOIN automatically joins columns with the same name in both tables.

Algorithm / Procedure:
1. Create team and player tables and insert the supplied records.
2. Use LEFT JOIN with team on the left so teams without players remain in the output.
3. Use CROSS JOIN to produce every team/player pair.
4. Use NATURAL JOIN to match rows through the common team_id column.

Program / Queries / Commands:
```
CREATE TABLE team (     team_id INT,
team_name VARCHAR(20) );

INSERT INTO team (team_id, team_name)
VALUES (1, 'Eagles'), (2, 'Tigers'), (3, 'Lions');

CREATE TABLE player (
player_name VARCHAR(30),
team_id INT,     age INT );

INSERT INTO player (player_name, team_id, age)
VALUES ('Asha', 1, 24), ('Meera', 1, 22), ('Arjun', 2, 27);

-- Task 1
SELECT t.team_name, p.player_name
FROM  team  AS  t LEFT
JOIN player AS p
    ON t.team_id = p.team_id
ORDER BY t.team_name, p.player_name;

-- Task 2
SELECT t.team_name, p.player_name
FROM team AS t
CROSS JOIN player AS p
ORDER BY t.team_name, p.player_name;

-- Task 3

SELECT p.player_name, t.team_name
FROM player AS p
NATURAL JOIN team AS t ORDER
BY p.player_name;
```

Input:

Teams: Eagles/1, Tigers/2, Lions/3. Players: Asha/1, Meera/1, Arjun/2.

Output:

Expected output (column spacing and display format may differ in your SQL tool):

Task 1: team_name	player_name
Eagles	Asha
Eagles	Meera
Lions	NULL
Tigers	Arjun
Task 2: team_name	player_name
Eagles	Arjun
Eagles	Asha
Eagles	Meera
Lions	Arjun
Lions	Asha
Lions	Meera
Tigers	Arjun
Tigers	Asha
Tigers	Meera
Task 3: player_name	team_name
Arjun	Tigers
Asha	Eagles
Meera	Eagles

Result:

The SQL statements for Experiment 11 were prepared to perform the stated tasks. On executing the setup and queries successfully, the expected results shown above should be obtained.

Viva-Voce Questions:

1. What is the difference between LEFT JOIN and INNER JOIN?
2. What is a CROSS JOIN?
3. What does NATURAL JOIN use to match tables?
4. Why does the Lions row show NULL for player_name?

Lab Assignment 12: Using a Subquery

Aim:

Use a subquery with IN to find students who are members of both clubs, count club members, and sort club names.

Software / Tools Required:

SQL database system (such as MySQL, PostgreSQL, or SQLite where supported), SQL editor/IDE, and a computer.

Theory:

A subquery is a query nested inside another SQL query. IN checks whether a value belongs to the result set returned by the subquery. COUNT(*) counts rows.

Algorithm / Procedure:

1. Create club_a and club_b and insert their supplied names.
2. Use a subquery after IN to find names from club_a that also occur in club_b.
3. Count the rows in club_b and alias the result members.
4. Select all names from club_a in alphabetical order.

Program / Queries / Commands:

```
CREATE TABLE club_a (     student_name
VARCHAR(30)
);

INSERT INTO club_a (student_name)
VALUES ('Asha'), ('Rohan'), ('Neha');

CREATE     TABLE     club_b     (
student_name VARCHAR(30)
);

INSERT INTO club_b (student_name)
VALUES ('Neha'), ('Pooja'), ('Meera');

-- Task 1
SELECT student_name FROM
club_a
WHERE student_name IN (
    SELECT student_name
    FROM club_b
);

-- Task 2
SELECT COUNT(*) AS members
FROM club_b;

-- Task 3
SELECT student_name
FROM club_a
ORDER BY student_name;
```

Input: club_a: Asha, Rohan, Neha. club_b: Neha, Pooja, Meera.

Output:

Expected output (column spacing and display format may differ in your SQL tool):

Task 1: student_name
Neha
Task 2: members
3
Task 3: student_name
Asha
Neha
Rohan

Result:

The SQL statements for Experiment 12 were prepared to perform the stated tasks. On executing the setup and queries successfully, the expected results shown above should be obtained.

Viva-Voce Questions:

1. What is a subquery?
2. What does IN check?
3. Can a subquery be used inside WHERE?
4. What does COUNT(*) count?

5. *Lab Assignment 13: UNION and INTERSECT*

Aim:
Combine names from two clubs, find common names, and include duplicate names using set operators.

Software / Tools Required:
SQL database system (such as MySQL, PostgreSQL, or SQLite where supported), SQL editor/IDE, and a computer.

Theory:
UNION combines result sets and removes duplicates. INTERSECT returns rows common to both result sets. UNION ALL combines result sets while retaining duplicates. The SELECT statements must return compatible columns.

Algorithm / Procedure:
1. Create club_a and club_b and insert the supplied names.
2. Use UNION and sort the unique combined names.
3. Use INTERSECT to find names in both clubs.
4. Use UNION ALL to keep repeated names and sort the result.

Program / Queries / Commands:
```
CREATE TABLE club_a (     student_name
VARCHAR(30) );

INSERT INTO club_a (student_name)
VALUES ('Asha'), ('Rohan'), ('Neha');

CREATE TABLE club_b (
student_name VARCHAR(30) );

INSERT INTO club_b (student_name)
VALUES ('Neha'), ('Pooja'), ('Meera');

-- Task 1
SELECT student_name FROM club_a
UNION
SELECT student_name FROM club_b
ORDER BY student_name;

-- Task 2
SELECT student_name FROM club_a
INTERSECT
SELECT student_name FROM club_b
ORDER BY student_name;

-- Task 3
SELECT student_name FROM club_a
UNION ALL
SELECT student_name FROM club_b
ORDER BY student_name;
```

Input:

club_a: Asha, Rohan, Neha. club_b: Neha, Pooja, Meera.

Output:

Expected output (column spacing and display format may differ in your SQL tool):

Task 1: student_name
Asha
Meera
Neha
Pooja
Rohan
Task 2: student_name
Neha
Task 3: student_name
Asha
Meera
Neha
Neha
Pooja
Rohan

Result:

The SQL statements for Experiment 13 were prepared to perform the stated tasks. On executing the setup and queries successfully, the expected results shown above should be obtained.

Viva-Voce Questions:

1. What is UNION used for?
2. How is UNION different from UNION ALL?
3. What does INTERSECT return?
4. What condition must the SELECT statements meet when using set operators?

5. *Lab Assignment 14: Creating and using a View*

Aim:
Create a view containing students with marks of 70 or more and retrieve rows from that view.

Software / Tools Required:
SQL database system (such as MySQL, PostgreSQL, or SQLite where supported), SQL editor/IDE, and a computer.

Theory:
A view is a named query that can be used like a virtual table. CREATE VIEW stores the query definition. SELECT can retrieve data from the view.

Algorithm / Procedure:
1. Create score and insert the four supplied student records.
2. Create the high_scores view with name and marks for marks >= 70.
3. Select all rows from high_scores.
4. Select only names from high_scores and sort them alphabetically.

Program / Queries / Commands:
```
CREATE TABLE score ( name
VARCHAR(30), subject VARCHAR(20),
marks INT
);

INSERT INTO score (name, subject, marks)
VALUES
    ('Asha', 'Maths', 78),
    ('Rohan', 'Maths', 64),
    ('Neha', 'Maths', 88),
    ('Kiran', 'Maths', 52);

-- Task 1
CREATE VIEW high_scores AS
SELECT name, marks
FROM score
WHERE marks >= 70;

-- Task 2
SELECT * FROM high_scores;

-- Task 3
SELECT name FROM high_scores ORDER
BY name;
```

Input:
Asha—78, Rohan—64, Neha—88, Kiran—52.

Output:

Expected output (column spacing and display format may differ in your SQL tool):

Task 2: name	marks
Asha	78
Neha	88
Task 3: name	
Asha	
Neha	

Result:

The SQL statements for Experiment 14 were prepared to perform the stated tasks. On executing the setup and queries successfully, the expected results shown above should be obtained.

Viva-Voce Questions:

1. What is a view in SQL?
2. Which command creates a view?
3. Does a view store a query definition?
4. How can rows be retrieved from a view?

5. *Lab Assignment 15: Indexed and Adding Row*

Aim:
Create an index on the city column, find customers in Delhi, and insert a new customer.

Software / Tools Required:
SQL database system (such as MySQL, PostgreSQL, or SQLite where supported), SQL editor/IDE, and a computer.

Theory:
An index is a database object that can help the DBMS find rows more efficiently. CREATE INDEX creates an index on one or more columns. INSERT adds a new row to a table.

Algorithm / Procedure:
1. Create customer with customer_id as primary key and insert the four setup customers.
2. Create the idx_city index on the city column.
3. Select customers whose city is Delhi and sort by name.
4. Insert customer 5, Arjun, from Pune, then display the table.

Program / Queries / Commands:
```
CREATE TABLE customer (    customer_id
INT PRIMARY KEY,     name VARCHAR(30)
NOT NULL,     city VARCHAR(20)
);

INSERT INTO customer (customer_id, name, city)
VALUES
    (1, 'Kiran', 'Kochi'),
    (2, 'Deepak', 'Delhi'),
    (3, 'Lata', 'Mumbai'),
    (4, 'Asha', 'Delhi');

-- Task 1
CREATE INDEX idx_city
ON customer (city);

-- Task 2
SELECT name FROM customer
WHERE city = 'Delhi'
ORDER BY name;

-- Task 3
INSERT INTO customer (customer_id, name, city)
VALUES (5, 'Arjun', 'Pune');

SELECT * FROM customer ORDER
BY customer_id;
```

Input:
Setup customers: Kiran/Kochi, Deepak/Delhi, Lata/Mumbai, Asha/Delhi. New customer: 5/Arjun/Pune.

Output:

Expected output (column spacing and display format may differ in your SQL tool):

Task 2: name		
Asha		
Deepak		
Task 3: customer_id	name	city
1	Kiran	Kochi
2	Deepak	Delhi
3	Lata	Mumbai
4	Asha	Delhi
5	Arjun	Pune

Result:

The SQL statements for Experiment 15 were prepared to perform the stated tasks. On executing the setup and queries successfully, the expected results shown above should be obtained.

Viva-Voce Questions:

1. What is an index in DBMS?
2. Which command creates an index?
3. What is the purpose of a primary key?
4. What does NOT NULL mean?